

ASSESSMENT-1

SOLUTION

1. The Water Jug Problem

Given: a 4-gallon jug and a 3-gallon jug, neither marked, with a pump for filling and the ability to pour water out or between jugs. Goal: get exactly 2 gallons into the 4-gallon jug.

Problem Understanding

This is a state-space search problem. A state is written as (x, y) where x is the amount of water currently in the 4-gallon jug and y is the amount in the 3-gallon jug. The start state is $(0, 0)$ - both jugs empty - and the goal is any state where $x = 2$, regardless of y .

Method Selection

The problem is solved by systematically applying the allowed operators - fill a jug completely, empty a jug completely, or pour from one jug into the other until either the source is empty or the destination is full - and tracking the resulting state after each move. This is essentially an uninformed search (state transitions applied step by step) rather than a calculation, so the method is a sequence of valid moves rather than a formula.

Solution Process

Operators available at any state (x, y) :

- Fill 4-gallon jug: $(x, y) \rightarrow (4, y)$
- Fill 3-gallon jug: $(x, y) \rightarrow (x, 3)$
- Empty 4-gallon jug: $(x, y) \rightarrow (0, y)$
- Empty 3-gallon jug: $(x, y) \rightarrow (x, 0)$
- Pour 4-gallon jug into 3-gallon jug until 3-gallon is full or 4-gallon is empty
- Pour 3-gallon jug into 4-gallon jug until 4-gallon is full or 3-gallon is empty

Step	Action	4-gallon jug	3-gallon jug
0	Start	0	0
1	Fill the 4-gallon jug from the pump	4	0

2	Pour 4-gallon jug into 3-gallon jug	1	3
3	Empty the 3-gallon jug on the ground	1	0
4	Pour the 1 gallon from the 4-gallon jug into the 3-gallon jug	0	1
5	Fill the 4-gallon jug from the pump	4	1
6	Pour 4-gallon jug into 3-gallon jug (only 2 gallons fit, since it already has 1)	2	3

Interpretation of Result

After step 6 the 4-gallon jug holds exactly 2 gallons and the goal state (2, 3) is reached. The solution needed 6 moves and only used the allowed operations of filling, emptying and pouring between the two jugs.

2. Mars Rover as an Intelligent Agent

A Mars rover is analysed here as a rational agent, using the PEAS framework (Performance, Environment, Actuators, Sensors) and by identifying a suitable agent architecture.

(a) Precepts of the agent

- Camera images and stereo/panoramic video of the surrounding terrain
- Readings from spectrometers and drills on soil and rock composition
- GPS/positioning and inertial data giving the rover's own location and tilt
- Obstacle-distance readings from proximity and hazard-avoidance sensors
- Atmospheric readings - temperature, dust levels, radiation
- Internal readings - battery/power level and wheel odometry

(b) Characterisation of the operating environment

- Partially observable - the rover only senses its immediate surroundings, not the whole of Mars
- Stochastic - wheel slippage, dust storms and hidden rocks make outcomes of actions uncertain

- Sequential - each move affects the rover's position and options for later moves
- Dynamic - weather and lighting change over time even while the rover is deciding
- Continuous - position, speed and sensor values vary continuously rather than in discrete steps
- Single-agent - the rover works toward NASA/ISRO mission goals on its own, without another rover competing against it

(c) Actions the agent can take

- Move forward, reverse, or turn by a given angle
- Extend the robotic arm and drill or scoop a sample
- Capture images and run onboard spectrometer scans
- Transmit collected data to an orbiter or directly to Earth
- Stop and reposition solar panels to recharge
- Re-plan a path when an obstacle is detected

(d) Evaluating agent performance

The performance measure combines: quantity and quality of scientific samples/data gathered, distance covered safely without damage or getting stuck, power consumed versus power available, number of hazards avoided, and how closely the rover meets mission time targets.

(e) Suitable agent architecture

A model-based, utility-based agent is the best fit. The rover cannot see the whole environment at once (partial observability), so it must keep an internal model/map of Mars built from past precepts. On top of that model it needs a utility function to choose between several safe paths or sample sites - for example trading off travel distance against scientific value and battery risk - rather than just reacting to the current percept. A simple reflex agent is not enough here because decisions depend on stored history and a comparison of possible outcomes.

3. 8-Queens Problem: Search Space Formulation

Goal: place 8 queens on an 8x8 board so that no two queens share a row, column, or diagonal.

Problem Understanding

Two queens attack each other if they are in the same row, same column, or same diagonal. The example configuration described in the question is invalid because the queen in the first column and the queen in the last column lie on the same diagonal.

Method Selection - Incremental Formulation

The 8-queens problem is best formulated incrementally, placing one queen per column, because this keeps the search space much smaller than considering every square on the board independently.

Problem Formulation

- Initial state: an empty chessboard with no queens placed
- States: any arrangement of 0 to 8 queens on the board, at most one queen per column, with no two queens attacking each other so far
- Actions: add a queen to any square of the leftmost empty column such that it is not attacked by any queen already on the board
- Transition model: returns the board with the new queen added in the chosen row of that column
- Goal test: 8 queens are on the board and none of them attack one another
- Path cost: not relevant here - every action costs the same, so any solution reaching the goal is equally acceptable (this is a constraint satisfaction search, not an optimisation)

Solution Process / Search Space Size

A naive formulation that allows any queen on any of the 64 squares gives a search space of 64 choose 8, which is enormous. Restricting placement to one queen per column and one per row cuts this down to $8! = 40,320$ permutations, and checking diagonal conflicts as each queen is placed prunes this further so a systematic search (for example backtracking/depth-first search) finds a valid solution quickly instead of testing all permutations blindly.

Interpretation of Result

The incremental, one-queen-per-column formulation is what makes the 8-queens problem tractable for search: instead of exploring roughly 4.3×10^9 raw board arrangements, the search

only has to explore at most 8^8 partial placements, and conflict checking at each step removes most of those before they are even expanded.

4. OLA Cab Booking Agent

Problem Understanding

The customer wants to move from a start location to a destination and must choose among several cab categories - mini, micro, sedan, shared, prime - based on comfort and cost preference. This is a well-defined problem: a clear initial state (pickup point), a clear goal (reach the destination), and a known set of actions (select a cab type, follow a route).

Method Selection

A simple problem-solving agent is appropriate: it formulates the goal (reach the destination comfortably/economically), formulates the problem (available cab types and routes), searches for the best option, and then executes the booking. This is a goal-based agent that uses search internally rather than a purely reflex agent, since it must compare multiple route/cab combinations before acting.

Solution Process – Pseudocode

function OLA-CAB-AGENT(pickup, destination, preference):

```
    problem <- FORMULATE-PROBLEM(pickup, destination)
```

```
    cabs    <- GET-AVAILABLE-CABS(pickup)    // mini, micro, sedan, shared, prime
```

```
    cabs    <- FILTER-BY-PREFERENCE(cabs, preference)
```

```
    route   <- SEARCH(problem)
```

```
    best_cab <- SELECT-BEST(cabs, route)
```

```
    BOOK(best_cab, route)
```

```
    return best_cab, route
```

Interpretation of Result

The pseudocode shows the agent moving through the standard problem-solving cycle - formulate goal, formulate problem, search, execute - which matches how the OLA app narrows

down many cab options to the one route/cab combination that best satisfies the customer's stated comfort and cost preference.

5. Uniform Cost Search for the Delivery Network

Uniform Cost Search (UCS) expands the least-cost frontier node first, using a priority queue ordered by cumulative path cost $g(n)$, and guarantees the optimal (least-cost) path from the start warehouse S to the goal warehouse G, provided all edge costs are non-negative.

Problem Understanding

The delivery network is a weighted graph where nodes are warehouses and edge weights represent the combined fuel and time cost of that route. Using the routes and costs given in the network diagram for this question, the warehouses and connecting routes are:

Route	Cost
S -> A	2
S -> B	5
A -> B	1
A -> C	8
B -> D	2
D -> C	1
D -> G	7
C -> G	3

Note: substitute the exact edge costs printed in your copy of the network diagram if they differ from the ones above - the UCS procedure below is applied the same way regardless of the numbers.

Method Selection

UCS is chosen instead of plain BFS because edge costs are not uniform (fuel + time varies per route), so the algorithm must track and compare cumulative path cost $g(n)$ rather than just the number of hops.

Solution Process

Frontier is a priority queue ordered by cumulative cost $g(n)$. At every step the lowest-cost node is removed and expanded; if a cheaper path to an already-frontier node is found, it replaces the older, costlier entry.

Node expanded	Path so far	Cumulative cost $g(n)$	Frontier after expansion
S	S	0	A(2), B(5)
A	S-A	2	B(3 via A), C(10), B(5) discarded
B	S-A-B	3	D(5), C(10)
D	S-A-B-D	5	C(6 via D), G(12), C(10) discarded
C	S-A-B-D-C	6	G(9 via C), G(12) discarded
G (goal)	S-A-B-D-C-G	9	search stops - goal removed from frontier

Accuracy of Calculation

Checking every complete route from S to G against the cost table confirms 9 is the minimum:

- $S-A-B-D-C-G = 2 + 1 + 2 + 1 + 3 = 9$
- $S-A-B-D-G = 2 + 1 + 2 + 7 = 12$
- $S-A-C-G = 2 + 8 + 3 = 13$
- $S-B-D-C-G = 5 + 2 + 1 + 3 = 11$
- $S-B-D-G = 5 + 2 + 7 = 14$

Interpretation of Result

The least-cost path from warehouse S to warehouse G is $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow G$, with a total delivery cost of 9 units of fuel and time. This is the route the logistics company should schedule, since every other path through the network costs more, even though some of those paths use fewer warehouse stops.